Experiment No: 08

Experiment Title:

i. Study & Implementation of Views in SQL with Suitable Examples
ii. Study & Implementation of different types of constraints.
   - Not null, Unique, Check
   - Primary Key, Foreign Key
   - Default

Topics to be covered:

- Views (Creation, Retrieval, Update, Deletion)
- Different types of constraints (Not null, Unique, Check, Primary Key, Foreign Key, Default)

Views (Creation, Retrieval, Update, Deletion)

1. **Creating a View**

Syntax:

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Example:

```
CREATE VIEW IT_Employees AS
SELECT employee_id, first_name, salary
FROM employees
WHERE department = 'IT';
```

Sample employees Table:

employee_id	first_name	salary	department
1	Alice	60000	IT
2	Bob	70000	HR
3	Carol	75000	IT
4	David	50000	Finance

View IT_Employees Output:

employee_id	first_name	salary
1	Alice	60000
3	Carol	75000

2. **Querying (Retrieving Data) from a View**

Syntax:

```
SELECT * FROM view_name;
-- or
SELECT column1 FROM view_name WHERE condition;
```

Example:

```
SELECT * FROM IT_Employees;
```

Output:

employee_id	first_name	salary
1	Alice	60000
3	Carol	75000

3. **Updating Data Through a View**

Views can be updatable **only if**:

- It is based on a single table.
- It doesn't use GROUP BY, DISTINCT, joins, or aggregate functions.
- The view includes the primary key.

Syntax:

```
UPDATE view_name
SET column = value
WHERE condition;
```

Example:

```
UPDATE IT_Employees
SET salary = salary + 2000
WHERE employee_id = 1;
```

Effect:

- In the **employees** table, Alice's salary is updated from **60000 → 62000**.

4. **Inserting Data Through a View**

Allowed only if the view is **updatable** and includes all NOT NULL columns.

Syntax:

```
INSERT INTO view_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

This is rare with views and generally avoided. Use with caution.

Example:

```
INSERT INTO it_employees (employee_id, first_name, salary)
VALUES (100, 'rafiq', 90000);
```

Output:

`it_employees table`

employee_id	first_name	salary
1	Alice	60000
3	Carol	75000

`employees Table`

employee_id	first_name	salary	department
1	Alice	62000	IT
2	Bob	70000	HR
3	Carol	75000	IT
4	David	50000	Finance
100	rafiq	90000	*NULL*

5. **Deleting Records Through a View**

Syntax:

```
DELETE FROM view_name
WHERE condition;
```

Example:

```
DELETE FROM IT_Employees
WHERE employee_id = 3;
```

- This would delete Carol's record **from the base employees table**, not just from the view.

6. **Dropping a View**

Syntax:

```
DROP VIEW view_name;
```

Example:

```
DROP VIEW IT_Employees;
```

- This removes the **view definition**. The base table (employees) remains unaffected.

Advantages of Views

- Simplify complex SQL queries.
- Enhance data security by restricting access to sensitive columns.

- Allow different users to see different data representations.
- Provide backward compatibility during database restructuring.

Limitations of Views

- Cannot have indexes (except materialized views).
- Some views are not updatable.
- Performance can degrade with complex views and joins.

7. **Check View Definition**

To view the SQL that defines a view:

Syntax (for most RDBMS like MySQL, PostgreSQL):

```
SHOW CREATE VIEW view_name;
-- or
SELECT definition FROM information_schema.views WHERE
table_name = 'view_name';
```

8. **Replacing or Altering a View**

If you want to change the view:

Syntax:

```
CREATE OR REPLACE VIEW view_name AS
SELECT ...
```

Example:

```
CREATE OR REPLACE VIEW IT_Employees AS
SELECT employee_id, first_name, salary, department
FROM employees
WHERE department = 'IT';
```

Summary Table:

Operation	SQL Command Example
Create View	CREATE VIEW view_name AS SELECT ...
Query View	SELECT * FROM view_name
Update via View	UPDATE view_name SET ... WHERE ...
Delete via View	DELETE FROM view_name WHERE ...
Drop View	DROP VIEW view_name
Replace View	CREATE OR REPLACE VIEW view_name AS SELECT ...
Check Definition	SHOW CREATE VIEW view_name or query information_schema

Lab Tasks-1:

Sample Table: Employees

employee_id	first_name	last_name	salary	hire_date	department	date_of_birth	gender	city
1	John	Doe	60000	2020-01-15	IT	1985-08-12	M	New York
2	Jane	Smith	70000	2019-03-23	HR	1990-05-22	F	Chicago
3	Tom	Brown	75000	2021-07-01	Sales	1988-12-04	M	Boston
4	Alice	Green	85000	2018-11-05	IT	1987-07-19	F	Miami
5	Bob	White	50000	2022-09-12	Marketing	1995-03-30	M	Dallas

Tasks (Views Only)

1. Create a view named **High_Salary_Employees** for employees earning **above 70,000**.
2. Display all records from the **High_Salary_Employees** view.
3. Create a view named **IT_Dept_View** that shows all employees working in the **IT department**.
4. Retrieve only the **first_name, last_name, and salary** from **IT_Dept_View**.
5. Update the **salary** of an employee (e.g., Alice Green) using the **IT_Dept_View**.
6. Create a view named **City_View** that shows **employee_id, first_name, city** for all employees in **New York or Chicago**.
7. Delete an employee record from the **High_Salary_Employees** view and observe the effect on the base table.
8. Create a view named **Dept_Salary_View** that displays **first_name, department, salary** of employees whose salary is between **60000 and 80000**.
9. Replace the **High_Salary_Employees** view to include an extra column **department** using CREATE OR REPLACE VIEW.
10. Drop the **City_View** and verify its deletion using SHOW CREATE VIEW or information_schema.views.

Lab Task-2:

Sample Table: Customers

CustomerID	Name	City	Age	TotalPurchase
1	John Smith	New York	28	1200.50
2	Sarah Johnson	Los Angeles	34	850.75
3	Michael Lee	Chicago	22	640.00
4	Emma Wilson	Houston	29	980.10
5	Daniel Kim	Phoenix	41	1500.00
6	Olivia Brown	Dallas	36	1330.20
7	James Davis	New York	27	700.00
8	Emily Garcia	Chicago	31	920.75
9	David Anderson	Los Angeles	39	1010.00
10	Sophia Martinez	Houston	24	610.00

Tasks (Views Only)

1. Create a view named **High_Value_Customers** for customers whose **TotalPurchase > 1000**.
2. Display all records from **High_Value_Customers**.
3. Create a view named **Chicago_Customers** that shows only **Name, City, Age** of customers from **Chicago**.
4. Create a view named **Senior_Customers** for customers aged **over 30**, showing **Name, Age, and TotalPurchase**.
5. Update the **TotalPurchase** of a customer using the **High_Value_Customers** view.
6. Create a view named **City_Purchase_View** to display **City** and the **average TotalPurchase** of customers (hint: use aggregation).
7. Replace the **Senior_Customers** view to include an extra column **City** using CREATE OR REPLACE VIEW.
8. Create a view named **Houston_Customers_View** showing all customers from **Houston** with their **Name and TotalPurchase**.
9. Delete a customer record (e.g., Sophia Martinez) using a view and observe the base table.
10. Drop all created views using the DROP VIEW command.

Different types of constraints (Not null, Unique, Check, Primary Key, Foreign Key, Default)

1. NOT NULL Constraint

Purpose: Ensures a column cannot have a NULL value.

Example:

```
CREATE TABLE Student (
    StudentID INT,
    Name VARCHAR(50) NOT NULL
);

-- Inserting values
INSERT INTO Student (StudentID, Name) VALUES (1, 'Alice');
-- Works fine

INSERT INTO Student (StudentID) VALUES (2);
-- ✕ Error: Column 'Name' cannot be null
```

Output:

StudentID	Name
1	Alice

2. UNIQUE Constraint

Purpose: Prevents duplicate values in a column.

Example:

```
CREATE TABLE Employee (
    EmpID INT,
    Email VARCHAR(100) UNIQUE
);

INSERT INTO Employee VALUES (1, 'alice@example.com');
INSERT INTO Employee VALUES (2, 'bob@example.com');
INSERT INTO Employee VALUES (3, 'alice@example.com');

-- ✕ Error: Duplicate entry 'alice@example.com' for key 'Email'
```

Output:

EmpID	Email
1	alice@example.com
2	bob@example.com

3. CHECK Constraint

Purpose: Restricts column values based on a condition.

Example:

```
CREATE TABLE StudentAge (
    ID INT,
    Age INT CHECK (Age >= 18)
);

INSERT INTO StudentAge VALUES (1, 20);
-- ✅ Works
INSERT INTO StudentAge VALUES (2, 16);
-- ❌ Error: CHECK constraint failed
```

Output:

ID	Age
1	20

4. PRIMARY KEY Constraint

Purpose: Uniquely identifies each record in a table (implies NOT NULL + UNIQUE).

Example:

```
CREATE TABLE Department (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50)
);

INSERT INTO Department VALUES (1, 'IT');
INSERT INTO Department VALUES (1, 'HR');
-- ❌ Error: Duplicate entry '1' for PRIMARY KEY
```

Output:

DeptID	DeptName
1	IT

5. FOREIGN KEY Constraint

Purpose: Links data between two tables using a relationship.

Example:

```
CREATE TABLE Department (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50)
);
```

```
CREATE TABLE Employee (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    DeptID INT,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);

INSERT INTO Department VALUES (1, 'IT');
INSERT INTO Employee VALUES (101, 'Alice', 1);
-- ✅ Works

INSERT INTO Employee VALUES (102, 'Bob', 2);
 -- ❌ Error: No matching DeptID in Department
```

Output:

EmpID	EmpName	DeptID
101	Alice	1

6. DEFAULT Constraint

Purpose: Assigns a default value when none is provided.

Example:

```
CREATE TABLE CityInfo (
    ID INT,
    City VARCHAR(50) DEFAULT 'Unknown'
);

INSERT INTO CityInfo (ID, City) VALUES (1, 'New York');
INSERT INTO CityInfo (ID) VALUES (2); -- Uses default value
```

Output:

ID	City
1	New York
2	Unknown

7. AUTO_INCREMENT

Purpose: Automatically generates unique numbers.

Example:

```
CREATE TABLE Persons (
    PersonID INT NOT NULL AUTO_INCREMENT,
    Name VARCHAR(50),
    PRIMARY KEY (PersonID)
);

INSERT INTO Persons (Name) VALUES ('Alice');
INSERT INTO Persons (Name) VALUES ('Bob');
```

Output:

PersonID	Name
1	Alice
2	Bob

Combined Table Example

```
CREATE TABLE Student (
    StudentID INT PRIMARY KEY AUTO_INCREMENT,
    Name VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Age INT CHECK (Age >= 18),
    City VARCHAR(50) DEFAULT 'Unknown',
    DeptID INT,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

LAB Task-1:

Sample Table: Student

StudentID	Name	Email	Age	City	DeptID

Tasks (Constraints Practice)

1. Create the Department table with DeptID as PRIMARY KEY.
2. Create the Student table using all six constraints.
3. Insert valid student records (minimum 3).
4. Try inserting a student without Name to test NOT NULL.
5. Try inserting two students with the same Email to test UNIQUE.
6. Try inserting a student with Age = 15 to test CHECK.
7. Insert a record without City and confirm the DEFAULT value is used.
8. Show all student data using SELECT *.
9. Add one more student linked to an existing department (tests FOREIGN KEY).
10. Try inserting a student with a non-existent DeptID (should fail due to FOREIGN KEY).

LAB Task-2:

Sample Table: Employee

EmpID	EmpName	Salary	Age	Email	DeptID

Tasks (Constraints Practice)

1. Create a Department table with columns: DeptID (PK) and DeptName.
2. Create an Employee table with:
   - `EmpID → PRIMARY KEY AUTO_INCREMENT`
   - `EmpName → NOT NULL`
   - `Email → UNIQUE`
   - `Age → CHECK (Age >= 21)`
   - `Salary → CHECK (Salary >= 20000)`
   - `DeptID → FOREIGN KEY referencing Department(DeptID)`
3. Insert valid records for 3 employees.
4. Insert an employee with missing EmpName to test NOT NULL.
5. Insert two employees with the same Email to test UNIQUE.
6. Insert an employee with Age = 19 to test CHECK.
7. Insert an employee without specifying DeptID (should fail if FK not nullable).
8. Insert one record with a valid department ID (FK works).
9. Display all data from both tables using a JOIN.
10. Delete a department that is linked to an employee (should fail due to FK dependency).